

UNIVERSIDAD DON BOSCO

Facultad de Ingeniería - Escuela de Ciencias de la Computación

Asignatura: Desarrollo de Software Propietario

Tema: Proyecto de Cátedra **Fecha de Entrega:** 20/10/2026	
Docente: Ing. Karens Medrano	**Grupo de Laboratorio:** 03

Integrante	**Carnet**
Michael Caleb Ortiz Meléndez	OM260275
Nathaly Ivania Martinez Guerrero	MG260121

Repositorio: https://github.com/caleb-bvm/DSPDesafio03

Contenido

Parte I. Documentación del proyecto 3
1. Descripción general 3
2. Modelo de datos 4
3. EquiposAPI 7
4. Seguridad 9
5. Concurrencia optimista 11
6. EquiposMVC 13
7. EquiposReportes 17
8. Puesta en marcha 17
9. Pruebas 17
Anexos **18**
Parte II. Preguntas probables en la defensa 26
Modelo y EF Core 26
API y reglas de negocio 27
Seguridad 28
Concurrencia 29
Cliente MVC 29
Reportes 30
Puesta en marcha 30

Si el índice aparece vacío, haga clic derecho sobre él y elija Actualizar campo.

Parte I. Documentación del proyecto

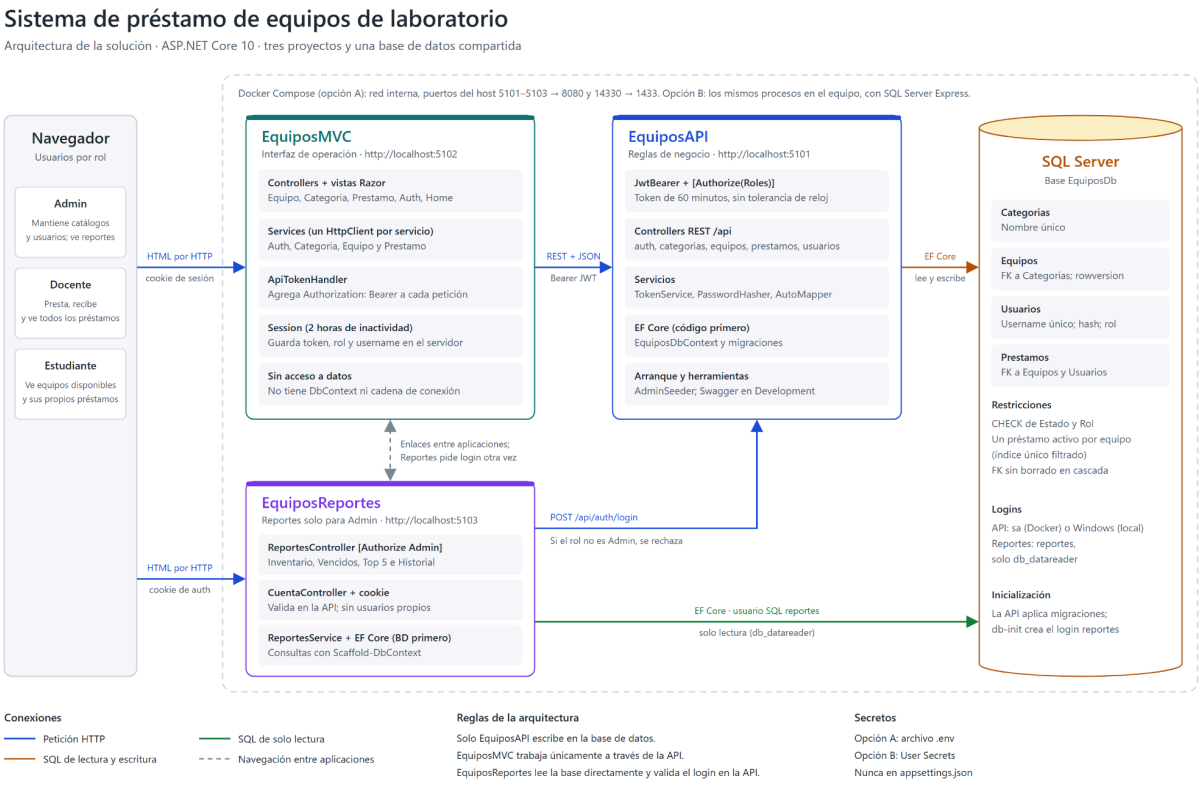
1. Descripción general

El laboratorio de computación les presta laptops, proyectores, kits de IoT y tarjetas Arduino a docentes y estudiantes. Antes todo se anotaba en una hoja de cálculo y muchas veces no se sabía quién tenía un equipo ni cuándo tenía que devolverlo. Por eso hicimos este sistema, que registra cada equipo, cada préstamo con su fecha de vencimiento y cada devolución.

Dividimos la solución en tres proyectos de ASP.NET Core 10, y los tres usan la misma base de datos SQL Server, `EquiposDb`:

Proyecto	Tecnología	Función
`EquiposAPI`	Web API, EF Core código primero, JWT	Aplica las reglas de negocio, emite los tokens y es el único proyecto que escribe en la base.
`EquiposMVC`	ASP.NET Core MVC, HttpClient, Session	Interfaz de operación para Admin, Docente y Estudiante. Todo lo hace llamando a la API.
`EquiposReportes`	ASP.NET Core MVC, EF Core base de datos primero	Cuatro reportes de solo lectura, exclusivos para el Admin.

Todo lo que se escribe en la base pasa por la API. EquiposMVC nunca abre una conexión a SQL Server: manda y recibe los datos por HTTP usando un token JWT. EquiposReportes sí se conecta a la base, pero con el login `reportes`, que solo pertenece al rol `db_datareader`. Además, antes de mostrar un reporte le pregunta a la API si quien entra es Admin.



2. Modelo de datos

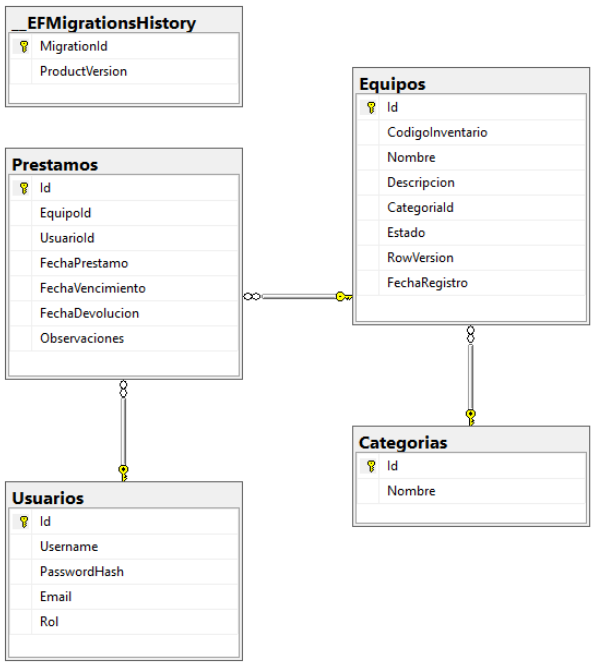
Escribimos el modelo como clases de C# y lo construimos con migraciones de EF Core, o sea, usamos el enfoque de código primero. La configuración de tablas, índices y relaciones está en `EquiposAPI/Data/EquiposDbContext.cs`.

Entidad	Campos	Restricciones
Categoria	Id, Nombre	Nombre único de hasta 100 caracteres. Cuatro categorías semilla con `HasData`: Laptops, Proyectores, Kits de IoT y Tarjetas Arduino.
Equipo	Id, CodigoInventario, Nombre, Descripcion, CategoriaId, Estado, FechaRegistro, RowVersion	CodigoInventario único. Estado se guarda como texto y una restricción CHECK solo admite Disponible, Prestado o Mantenimiento. RowVersion es el token de concurrencia.
Usuario	Id, Username, PasswordHash, Email, Rol	Username único. Una restricción CHECK limita Rol a Admin, Docente o Estudiante.
Prestamo	Id, EquipoId, UsuarioId, FechaPrestamo, FechaVencimiento, FechaDevolucion, Observaciones	FechaDevolucion queda nula hasta la devolución. Un índice único filtrado permite un solo préstamo activo por equipo.

Las relaciones son Categoria 1-N Equipo, Equipo 1-N Prestamo y Usuario 1-N Prestamo, y todas usan `DeleteBehavior.Restrict` (líneas 51, 79 y 81 del contexto). Con Restrict, SQL Server no deja borrar una categoría que tenga equipos ni un equipo que tenga préstamos, así el historial no se pierde por un borrado en cascada.

A Equipo le agregamos el campo `FechaRegistro`, porque el enunciado permite agregar campos. La API lo llena al crear el equipo y lo usamos para filtrar por fecha el reporte de inventario.

Creamos el esquema con tres migraciones que están en `EquiposAPI/Data/Migrations`: `Inicial`, `PrestamosUnoAMuchos` y `FechaRegistroEquipo`. A partir de ellas generamos el script `sql/EquiposDb.sql` con `dotnet ef migrations script --idempotent`. Cuando la API arranca, aplica las migraciones pendientes con `MigrateAsync()` (`AdminSeeder.cs`, línea 17).

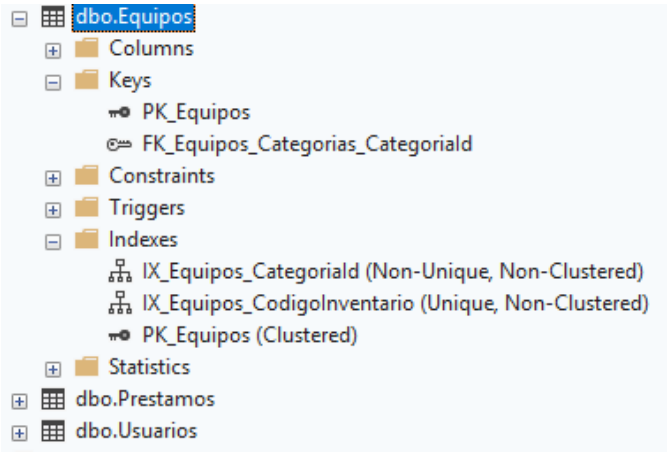


133 % No issues found

Results Messages

	Indice	Tipo	EsLlavePrimaria	EsUnico	Columnas
1	PK_Equipos	CLUSTERED	1	1	Id
2	IX_Equipos_CategoriaId	NONCLUSTERED	0	0	CategoriaId
3	IX_Equipos_CodigoInventario	NONCLUSTERED	0	1	CodigoInventario

	LlaveForanea	TablaOrigen	ColumnaOrigen	TablaReferenciada	ColumnaReferenciada	AlEliminar
1	FK_Equipos_Categorias_CategoriaId	Equipos	CategoriaId	Categorias	Id	NO_ACTION



133 % No issues found

Results Messages

	Id	Nombre
1	3	Kits de IoT
2	1	Laptops
3	2	Proyectores
4	4	Tarjetas Arduino

3. EquiposAPI

3.1 Endpoints y roles

Método	Ruta	Descripción	Rol
POST	`/api/auth/login`	Devuelve el token JWT	Anónimo
POST	`/api/auth/register`	Registra un usuario	Admin
GET	`/api/categorias`	Lista las categorías	Autenticado
POST, PUT, DELETE	`/api/categorias`	Mantenimiento de categorías	Admin
GET	`/api/equipos?texto=&categoriaId=&estado=`	Lista con filtros combinables y paginación	Autenticado
GET	`/api/equipos/{id}`	Detalle con el nombre de la categoría	Autenticado
POST, PUT, DELETE	`/api/equipos`	Mantenimiento de equipos	Admin
POST	`/api/prestamos`	Registra un préstamo	Admin, Docente
PUT	`/api/prestamos/{id}/devolucion`	Registra la devolución	Admin, Docente
GET	`/api/prestamos?soloVencidos=true`	Lista los préstamos según el rol del token	Autenticado
GET	`/api/usuarios`	Lista usuarios para elegir al prestatario	Admin, Docente

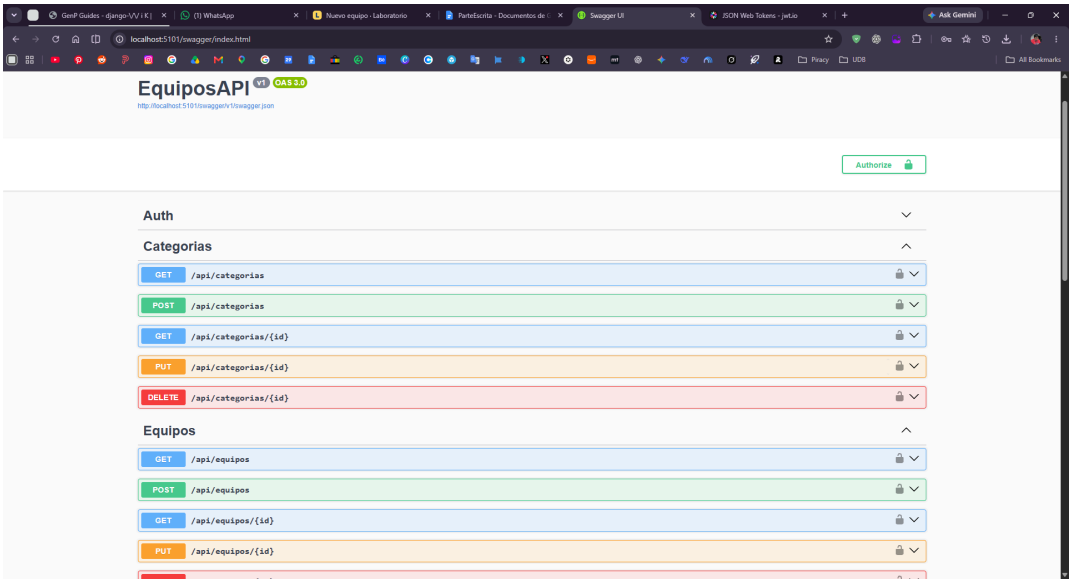
Todos los controladores llevan `[Authorize]`, y las acciones de escritura agregan `[Authorize(Roles = ...)]` según lo que dice la tabla. Al Estudiante la API solo le devuelve equipos disponibles, aunque pida otro estado en la URL (`EquiposController.cs`, línea 29).

3.2 DTOs, AutoMapper y Swagger

Los endpoints reciben y devuelven DTOs, nunca entidades. Los DTOs están en `EquiposAPI/Dtos` y tenemos uno distinto para crear, actualizar y listar. Por ejemplo, `EquipoActualizarDto` agrega `RowVersion` a los campos de `EquipoCrearDto`, y `EquipoListaDto` agrega `CategoriaNombre` y `FechaRegistro`. `UsuarioListaDto` no incluye `PasswordHash`.

Para convertir entre entidades y DTOs usamos AutoMapper 12.0.1, que es la versión de la Guía 11, con las reglas de `Mapping/MappingProfile.cs`. Swagger está en `/swagger` cuando el entorno es Development y trae un botón Authorize para probar con un token los endpoints protegidos.

Todas las respuestas de error tienen la forma `{ "message": "..." }`. Si ocurre una excepción no controlada, la API responde 500 con un mensaje genérico, sin mostrar detalles internos (`Program.cs`, línea 94).



Swagger con la lista de endpoints y el botón Authorize.

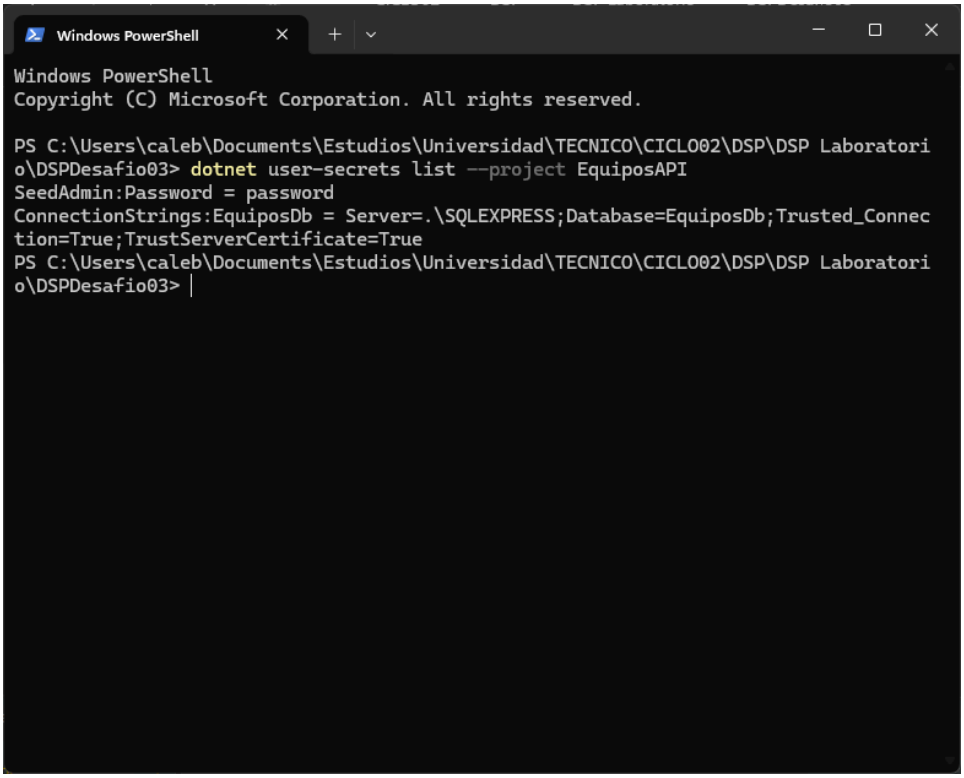
3.3 Reglas de negocio

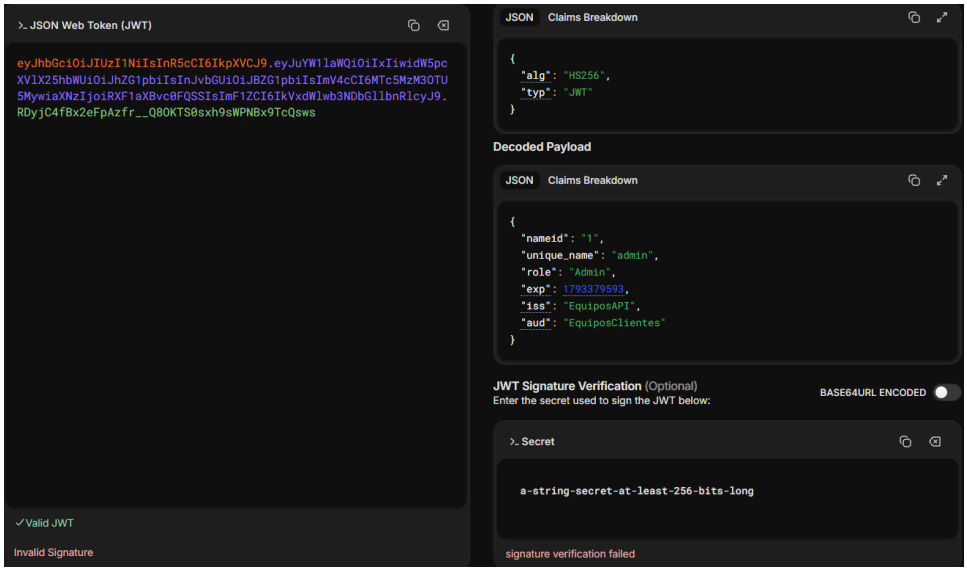
Regla	Dónde se aplica	Respuesta
Solo se presta un equipo Disponible	`PrestamosController.Crear`	409 con el estado actual del equipo
El préstamo y el cambio de estado a Prestado se guardan juntos	Un solo `SaveChangesAsync()`, línea 81	Se guardan los dos cambios o ninguno
El vencimiento va de hoy a 7 días después	`PrestamosController.cs`, línea 51	400 con mensaje
La devolución llena FechaDevolucion y deja el equipo Disponible	`PrestamosController.Devolver`	200; 409 si ya se había devuelto

Regla	Dónde se aplica	Respuesta
Un equipo con préstamos históricos no se elimina	`EquiposController.cs`, línea 159	409 con mensaje

4. Seguridad

- Las contraseñas se guardan como hash con `PasswordHasher<Usuario>`. En el login, `VerifyHashedPassword` compara lo que escribió el usuario con el hash, y si no coincide la API responde 401.
- El token trae los claims `nameid` (Id del usuario), `unique_name` (Username) y `role` (`TokenService.cs`, líneas 23 a 25). Dura 60 minutos, según `Jwt:MinutesToExpire`.
- Si la clave JWT tiene menos de 32 caracteres, la API no arranca (`Program.cs`, línea 23). Al validar el token tampoco hay tolerancia de tiempo: `ClockSkew = TimeSpan.Zero`.
- En el listado de préstamos, el Id del Estudiante sale del token (`PrestamosController.cs`, línea 32). El endpoint no recibe ningún parámetro de usuario.
- Si se intenta registrar un Username repetido, la API responde 409 (`AuthController.cs`, línea 45).
- Cuando la API arranca y la tabla Usuarios está vacía, `AdminSeeder` crea el usuario `admin` con la contraseña de `SeedAdmin:Password`. No sembramos usuarios con `HasData`.
- La cadena de conexión, la clave JWT y las contraseñas están en User Secrets. Con Docker van en variables de entorno que se leen del archivo `.env`, que Git ignora. En `appsettings.json` solo dejamos valores que no son secretos, como el emisor y la audiencia del token.





133 % No issues found Ln: 4, Ch: 23 TABS CRLF UTF-8

Results Messages

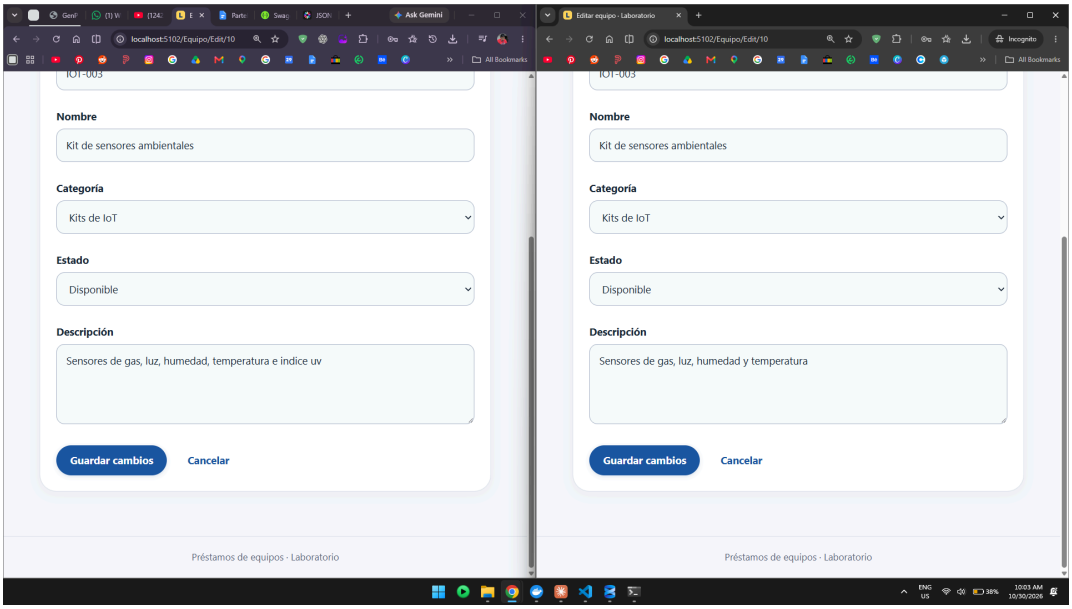
	Id	Username	PasswordHash	Email	Rol
1	1	admin	AQAAAAIAAYagAAAAEJaoLMeQeplcRWiCggV6d0cdB40Y+UPW0F...	admin@laboratorio.local	Admin
2	2	mlopez	AQAAAAIAAYagAAAAEMs7hDlBD7zB2YPCUs2alznbG+bJG6kR7l0...	mlopez@laboratorio.local	Docente
3	3	jhernandez	AQAAAAIAAYagAAAAEBxs/nwvl8Rlwy2gBu/eRltZZ1nd/Af+ncLVVHW...	jhernandez@laboratorio.local	Docente
4	4	acastro	AQAAAAIAAYagAAAAEF7xCkpKZOdyE5qZc1ILAXxhxloR/tDMQa/E...	acastro@laboratorio.local	Estudiante
5	5	dmartinez	AQAAAAIAAYagAAAAEMBsn/18uP9xCHN9mkQZUOBGCaXbAjSH...	dmartinez@laboratorio.local	Estudiante
6	6	sramirez	AQAAAAIAAYagAAAAENhUjrPLg1yj1kzs6ndpBN5SY4N6t6lo2vEwPJ...	sramirez@laboratorio.local	Estudiante
7	7	kflores	AQAAAAIAAYagAAAAEFO8K6HZkhU1/4dgYsD4tdHD4ML3mHTNfA...	kflores@laboratorio.local	Estudiante

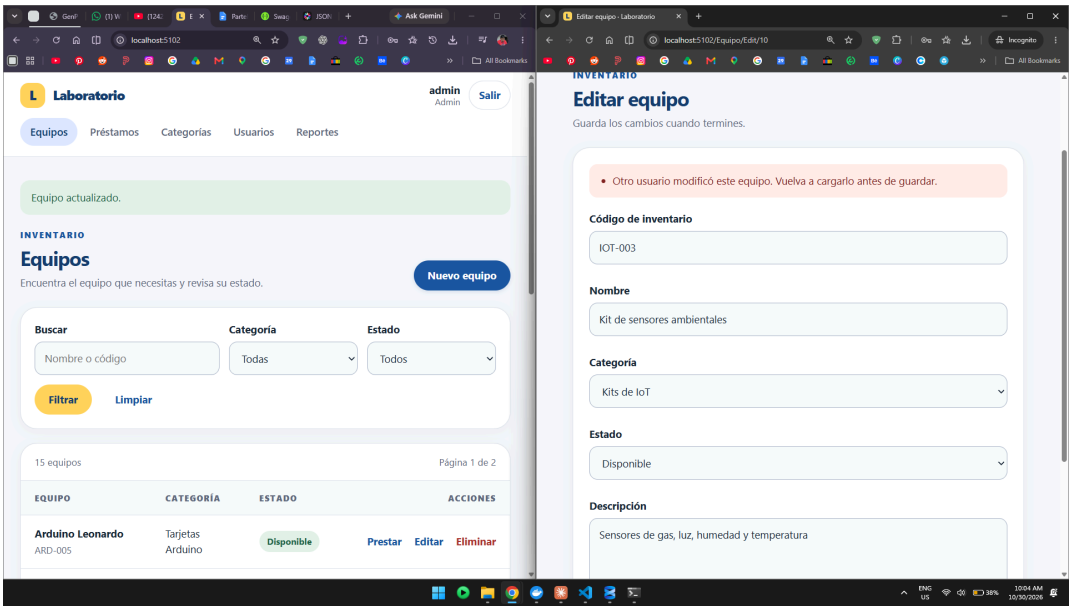
5. Concurrencia optimista

Equipo usa `RowVersion` como token de concurrencia (`IsRowVersion()`, línea 48 del contexto). SQL Server cambia ese valor en cada UPDATE. Así funciona todo el recorrido:

1. Cuando se abre Edit, el MVC le pide el equipo a la API y recibe la `rowVersion` en Base64.
2. La vista guarda ese valor en un campo oculto (`Views/Equipo/_Form.cshtml`, línea 9) y lo manda de vuelta al guardar.
3. La API lo asigna como valor original (`EquiposController.cs`, línea 127), y EF Core genera un `UPDATE ... WHERE Id = @id AND RowVersion = @versionLeida`.
4. Si alguien más guardó primero, la versión ya cambió, así que el UPDATE no afecta ninguna fila y EF lanza `DbUpdateConcurrencyException`. Entonces la API responde 409 (línea 141).
5. El MVC muestra el mensaje dentro del formulario y mantiene lo que el usuario había escrito (`EquipoController.cs`, línea 117).

Esto también protege los préstamos: al prestar un equipo cambia su estado, así que dos préstamos simultáneos del mismo equipo chocan con la RowVersion. Además, el índice único filtrado de Prestamos no deja que un equipo tenga dos préstamos activos.





6. EquiposMVC

El MVC no tiene autenticación propia. Cuando alguien inicia sesión, llama a `/api/auth/login` y guarda en Session el token, el rol y el Username. En la vista, el modelo y la API siempre usamos Username.

El token viaja en cada petición gracias a `ApiTokenHandler`, un `DelegatingHandler` que lee el token de la Session del usuario actual y agrega el encabezado `Authorization: Bearer` (`Services/ApiTokenHandler.cs`, línea 14). Lo registramos con `AddHttpMessageHandler` en cada cliente tipado, sin tocar `DefaultRequestHeaders`. La URL base de la API se lee de `Api:BaseUrl` en `appsettings.json` (`Program.cs`, línea 21).

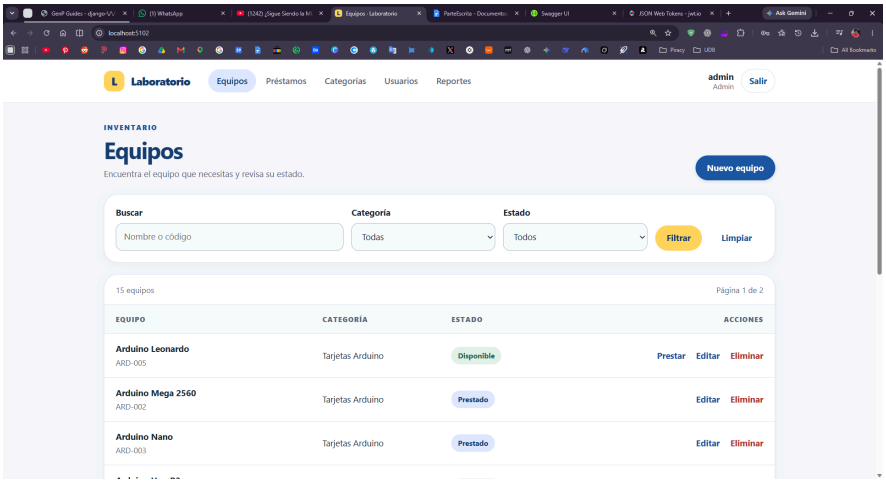
Tenemos un servicio y un controlador por recurso: equipos, categorías, préstamos y autenticación. Equipos y categorías tienen vistas Index, Create, Edit y Delete. En préstamos, Edit registra la devolución y Delete solo explica que los préstamos no se eliminan, para conservar el historial. Autenticación tiene las vistas Login y Register.

El menú cambia según el rol (`Views/Shared/_Layout.cshtml`, líneas 32 y 33). El Estudiante solo ve Equipos y Mis préstamos, el Docente ve Equipos y Préstamos, y el Admin ve además Categorías, Usuarios y Reportes.

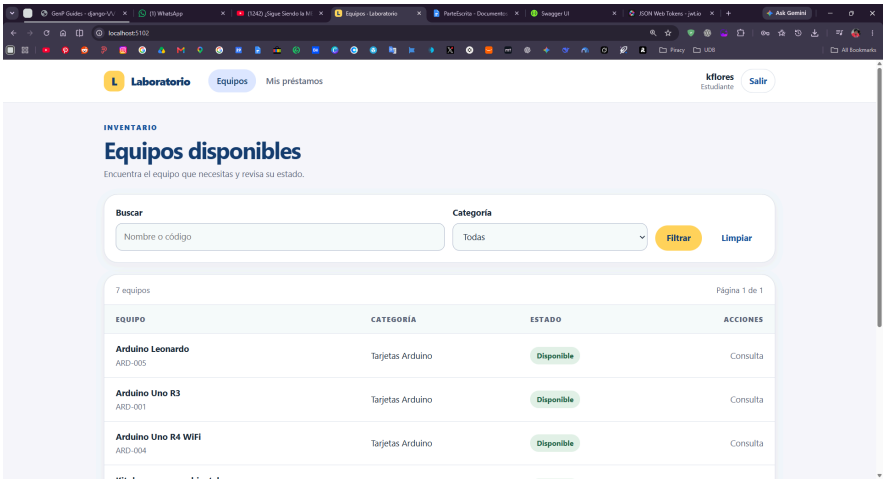
Código	Qué hace el cliente	Dónde
401	Limpia la Session y redirige al login con el aviso "Tu sesión venció"	`AppController.cs`, líneas 35 y 36
403	Muestra la vista de acceso denegado	`AppController.cs`, línea 41
409 y 400	Muestra el mensaje de la API en el formulario sin perder lo escrito	`ModelState.AddModelError` en cada acción

Código	Qué hace el cliente	Dónde
5xx o API apagada	Muestra un mensaje genérico, sin detalles internos	`AppController.cs`, línea 48

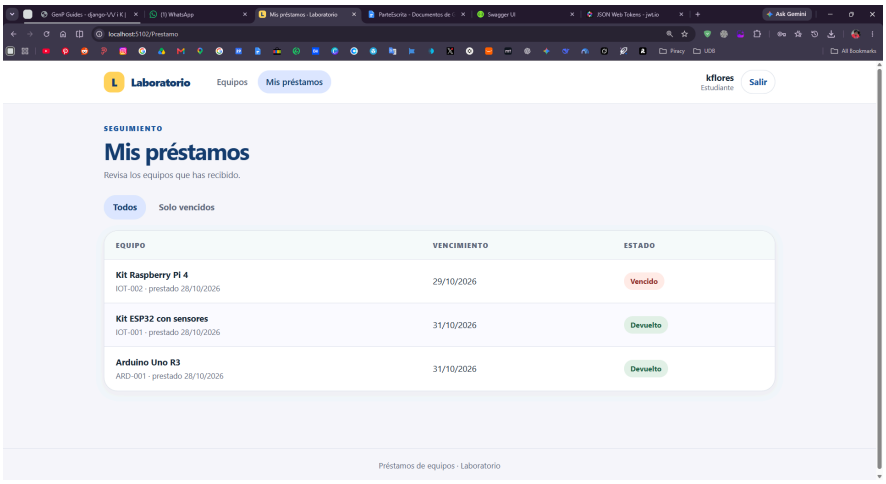
La vista Index de Equipos se puede filtrar por texto, categoría y estado, combinando los filtros, y muestra los resultados de 10 en 10.



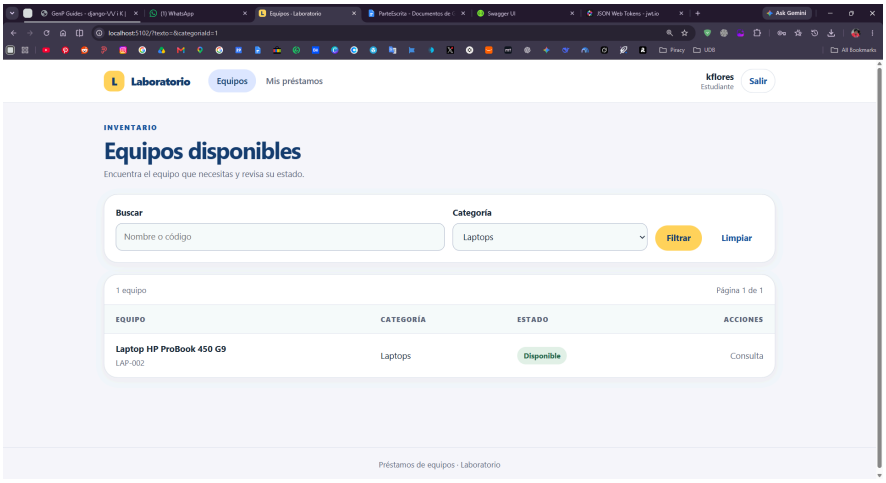
Vista de Administrador.



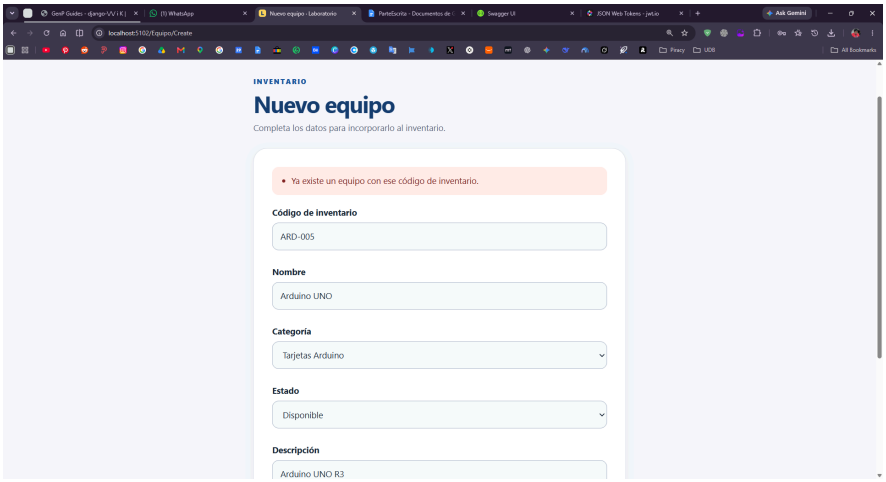
Vista de Equipos con el rol de Estudiante.



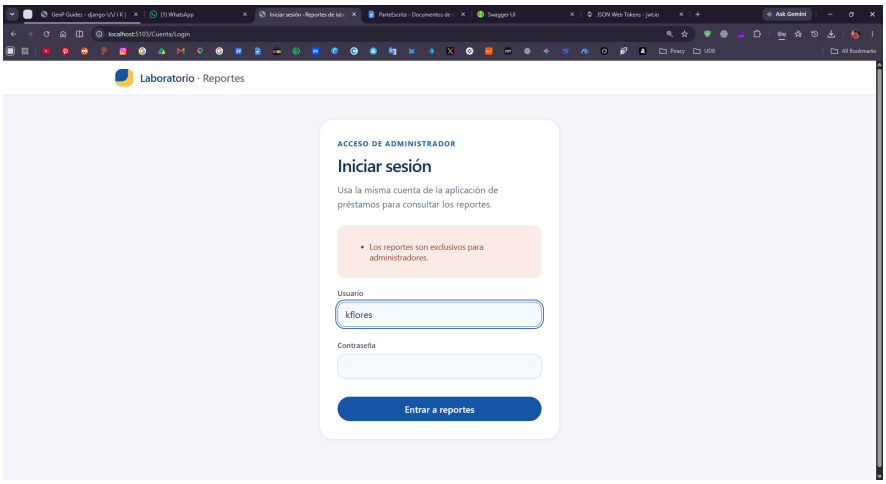
Vista de Mis préstamos de la estudiante Kflores.



Vista de Equipos filtrado desde el Rol estudiante.



Formulario de Create de Equipo con el mensaje 409 de código de inventario repetido y los datos escritos todavía en los campos.



Vista de acceso denegado.

7. EquiposReportes

Generamos el contexto y las entidades desde la base existente con `Scaffold-DbContext` (`EquiposReportes/Data`). El contexto no tiene `OnConfiguring`: lo registramos con inyección de dependencias en `Program.cs`, con la cadena `ConnectionStrings:EquiposReportes` de User Secrets.

Esa cadena usa el login SQL `reportes`, que se crea con `EquiposReportes/sql/crear_usuario_reportes.sql` y solo pertenece a `db_datareader`. Si el código intentara escribir algo, SQL Server lo rechazaría.

Para entrar, `CuentaController` manda el usuario y la contraseña a `/api/auth/login`. Si la API responde con rol Admin, crea una cookie con el nombre y el rol, que vence junto con el token. Con cualquier otro rol muestra un mensaje y no deja pasar. `ReportesController` lleva `[Authorize(Roles = "Admin")]`.

Todas las consultas de `Services/ReportesService.cs` usan `AsNoTracking()` y proyectan con `Select` a ViewModels, así que ninguna vista recibe entidades.

Reporte	Contenido	Filtros
Inventario por categoría	Cantidad de equipos por categoría y por estado	Categoría; fechas sobre FechaRegistro
Préstamos vencidos	Vencimiento anterior a hoy y sin devolución, con usuario y días de atraso	Categoría; fechas sobre FechaVencimiento
Top 5 equipos más prestados	Conteo de préstamos por equipo	Categoría; fechas sobre FechaPrestamo
Historial por usuario	Todos los préstamos de un Username	Username, categoría; fechas sobre FechaPrestamo

8. Puesta en marcha

Los pasos completos están en el README. En resumen, `.\iniciar.ps1` levanta todo con Docker, y `.\iniciar.ps1 -Local` usa el SQL Server Express de la computadora y User Secrets. En los dos casos el script crea las contraseñas que falten, arranca los tres proyectos, espera a que respondan y carga datos de demostración llamando a la API.

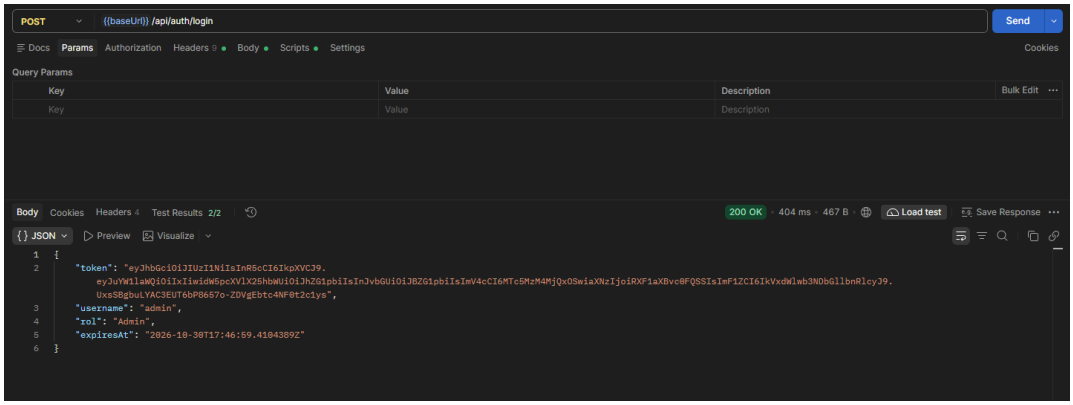
Para la opción local, la instancia tiene que aceptar autenticación de SQL Server, porque Reportes se conecta con el login `reportes`. Después de activarla en las propiedades del servidor hay que reiniciar el servicio SQL Server (SQLEXPRESS).

9. Pruebas

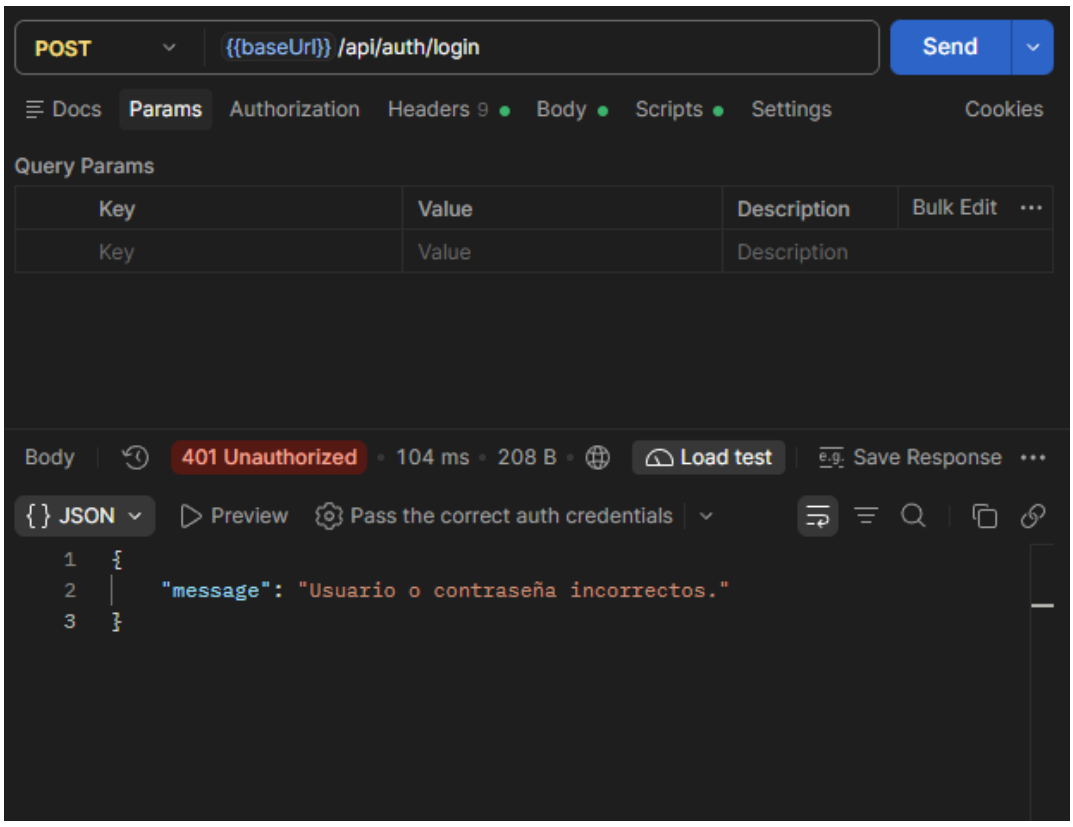
Estos son los 12 casos que se revisarán en la defensa. La colección `postman/EquiposLaboratorio.postman_collection.json` corre los casos 1 a 10 en orden, y los casos 11 y 12 los probamos en el MVC, en Reportes y en SSMS.

Nº	Caso	Resultado esperado	Evidencia
1	Login correcto	200 y un token con el claim de rol	**[CAPTURA 22]**
2	Login con contraseña incorrecta	401	**[CAPTURA 23]**
3	Endpoint protegido sin token	401	**[CAPTURA 24]**
4	Estudiante crea un equipo	403	**[CAPTURA 25]**
5	Username repetido	409	**[CAPTURA 26]**
6	Prestar un equipo ya Prestado	409	**[CAPTURA 27]**
7	Vencimiento a 15 días	400	**[CAPTURA 28]**
8	Eliminar equipo con historial	409	**[CAPTURA 29]**
9	Dos ediciones simultáneas	La segunda recibe 409	**[CAPTURA 30]**
10	Estudiante cambia parámetros en la URL	Solo ve sus propios préstamos	**[CAPTURA 31]**

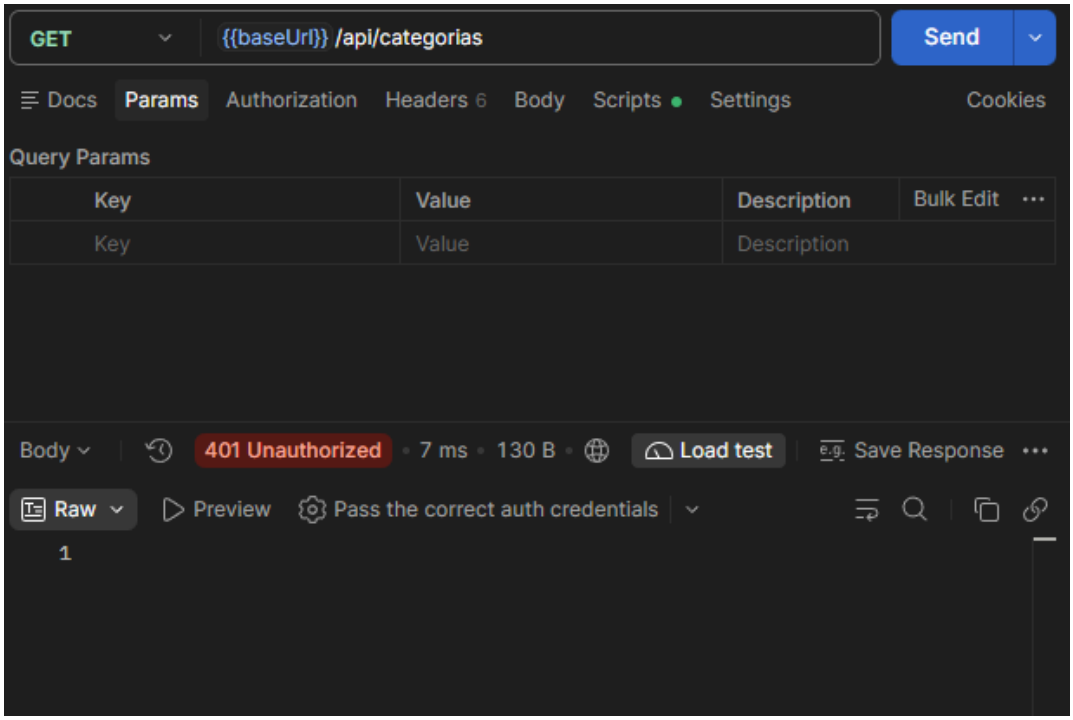
Anexos



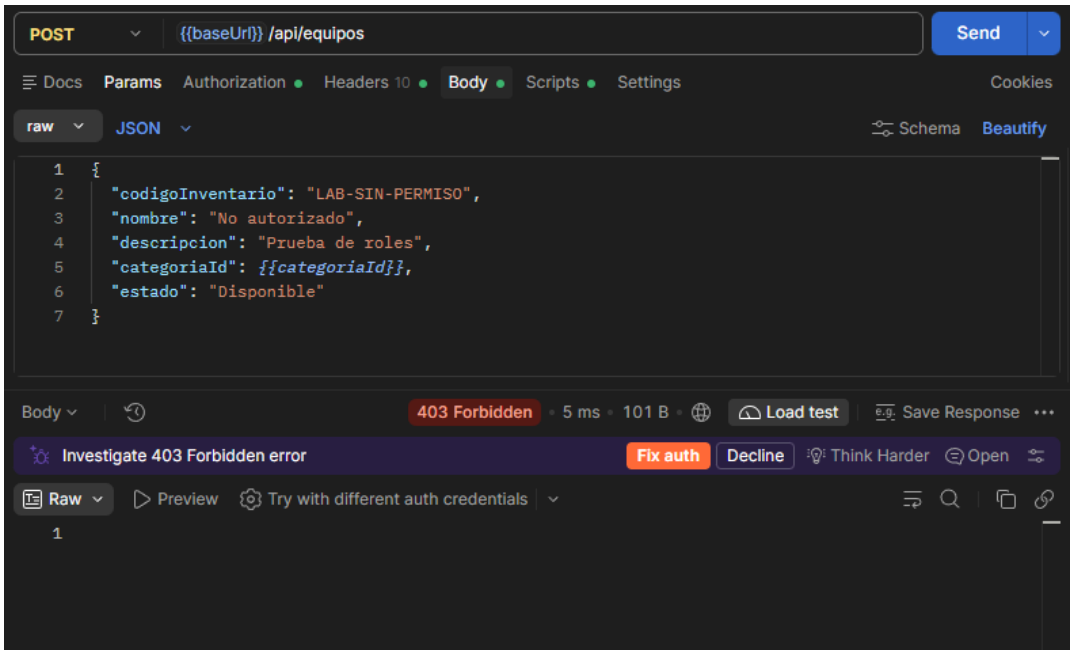
Captura 22.



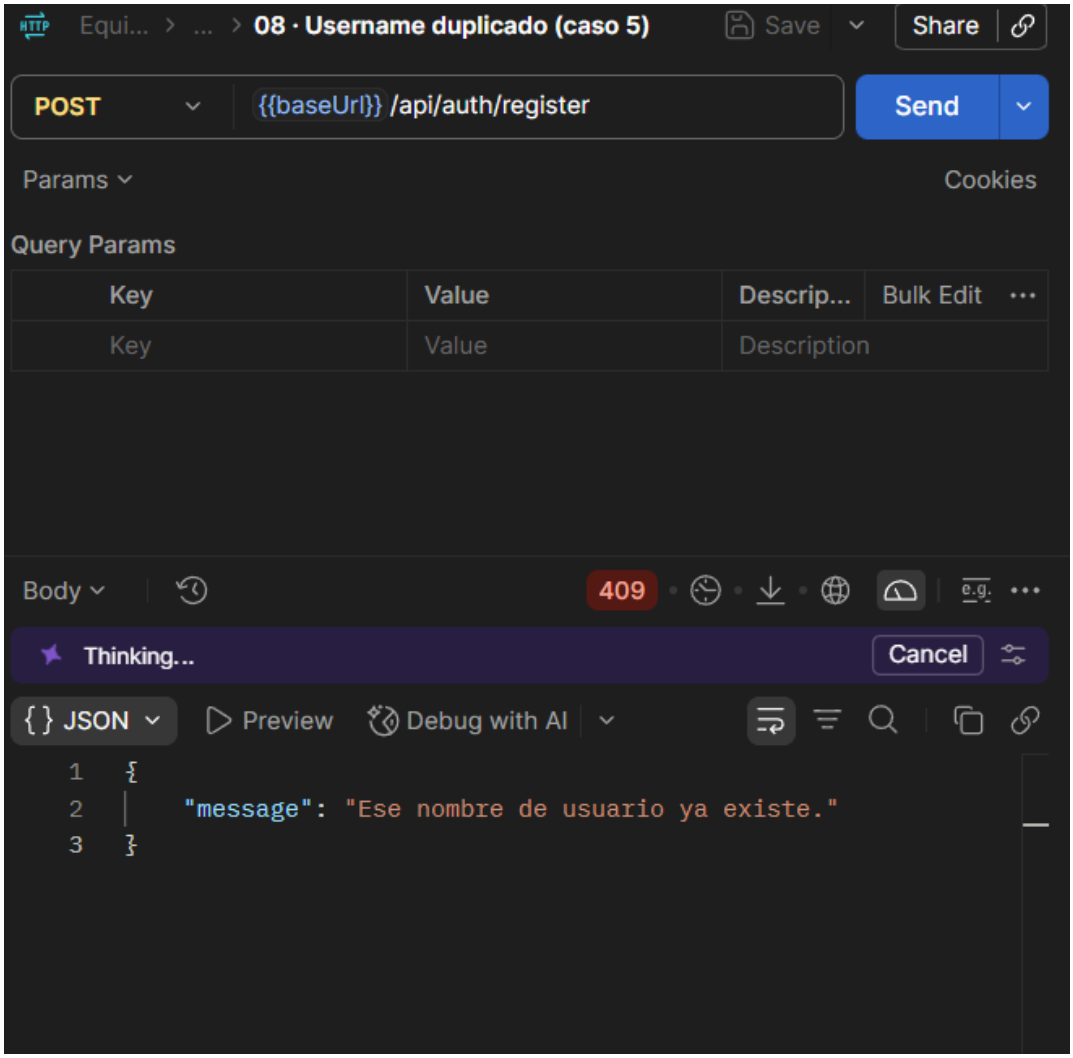
Captura 23.



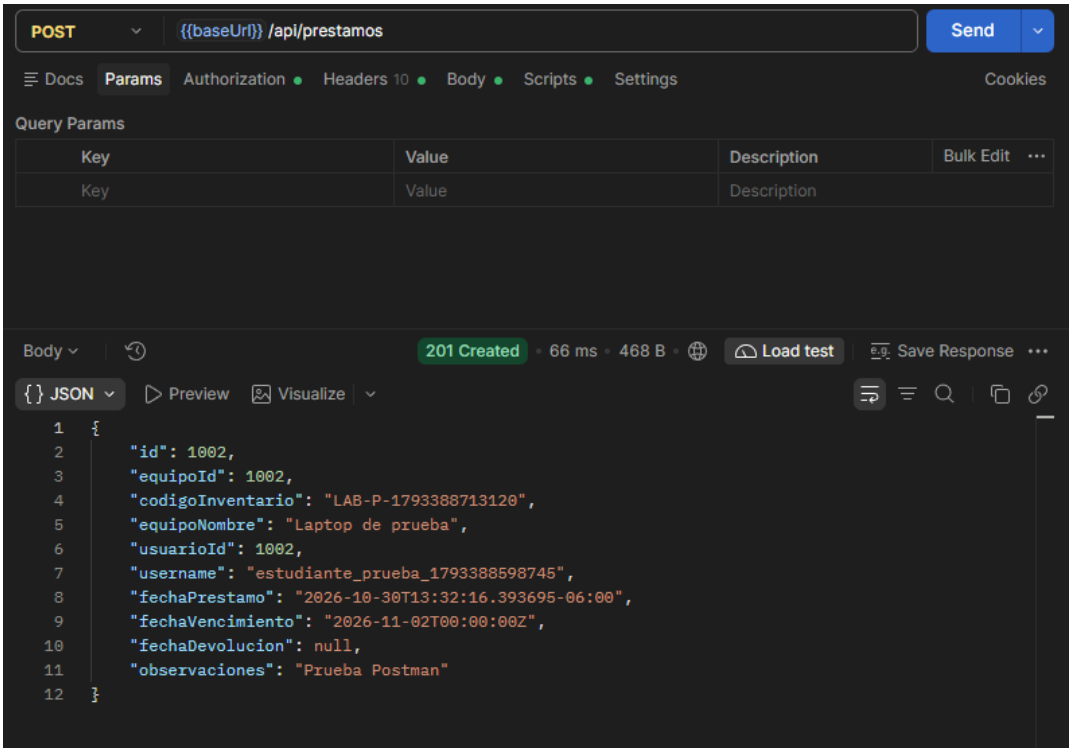
Captura 24.

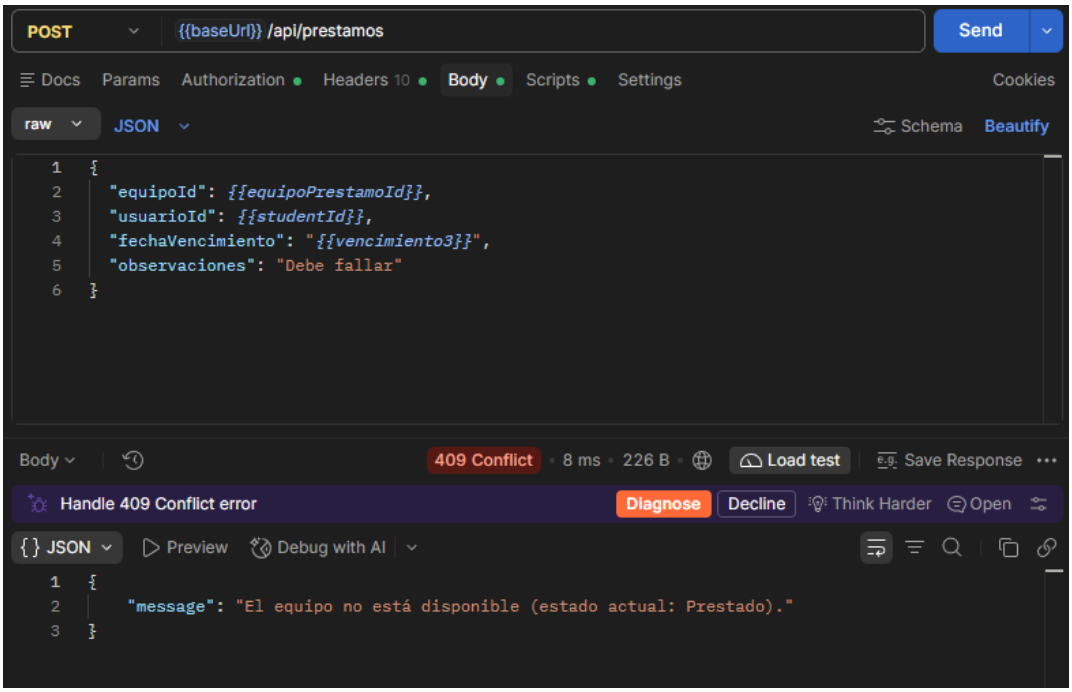


Captura 25.

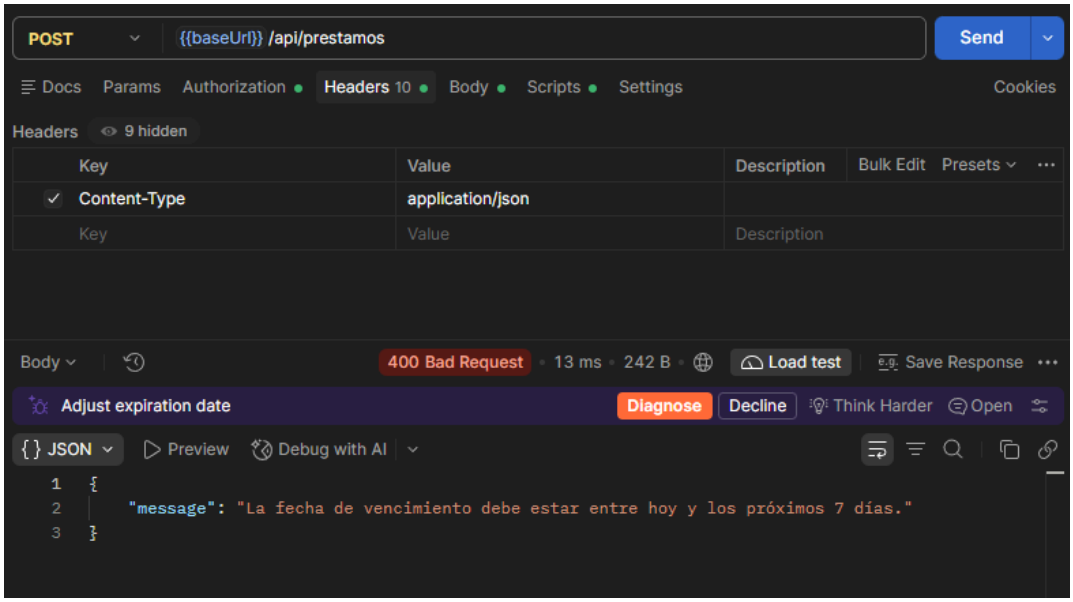

Equi... > ... > 08 · Username duplicado (caso 5)
Save
Share
POST
{{baseUrl}} /api/auth/register
Send
Params
Cookies
Query Params
Key
Value
Descrip...
Bulk Edit
Key
Value
Description
Body
409
Thinking...
Cancel
JSON
Preview
Debug with AI
1 {
2 "message": "Ese nombre de usuario ya existe."
3 }

Captura 26.

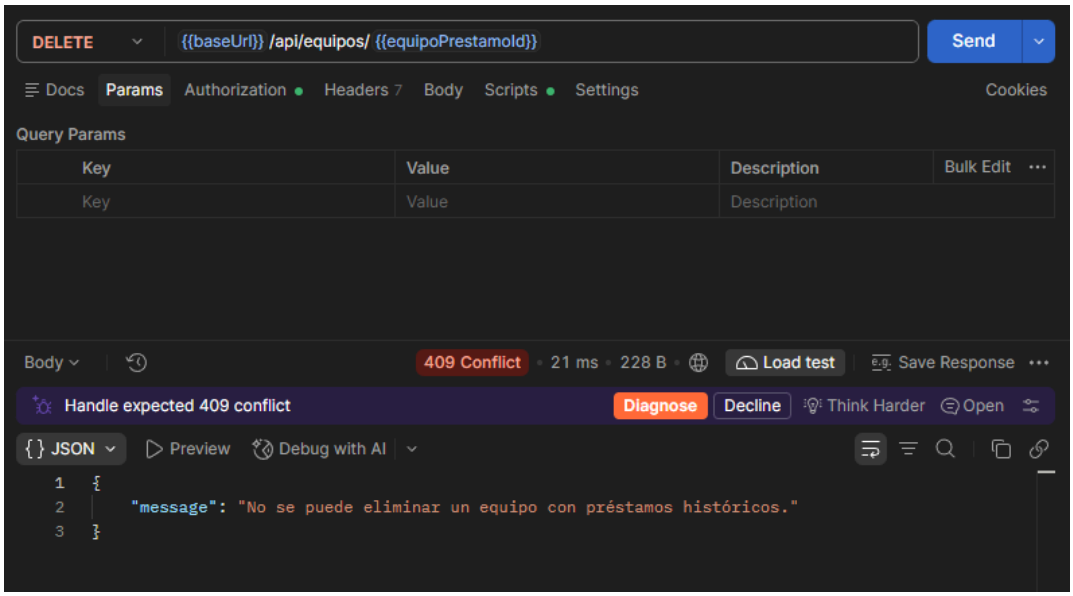

POST
{{baseUrl}} /api/prestamos
Send
Docs
Params
Authorization
Headers 10
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
201 Created
66 ms
468 B
Load test
Save Response
JSON
Preview
Visualize
{
"id": 1002,
"equipoId": 1002,
"codigoInventario": "LAB-P-1793388713120",
"equipoNombre": "Laptop de prueba",
"usuarioId": 1002,
"username": "estudiante_prueba_1793388598745",
"fechaPrestamo": "2026-10-30T13:32:16.393695-06:00",
"fechaVencimiento": "2026-11-02T00:00:00Z",
"fechaDevolucion": null,
"observaciones": "Prueba Postman"
}


POST
{{baseUrl}} /api/prestamos
Send
Docs
Params
Authorization
Headers 10
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"equipoId": {{equipoPrestamoId}},
"usuarioId": {{studentId}},
"fechaVencimiento": "{{vencimiento3}}",
"observaciones": "Debe fallar"
}
Body
409 Conflict
8 ms
226 B
Load test
Save Response
Handle 409 Conflict error
Diagnose
Decline
Think Harder
Open
JSON
Preview
Debug with AI
{
"message": "El equipo no está disponible (estado actual: Prestado)."
}

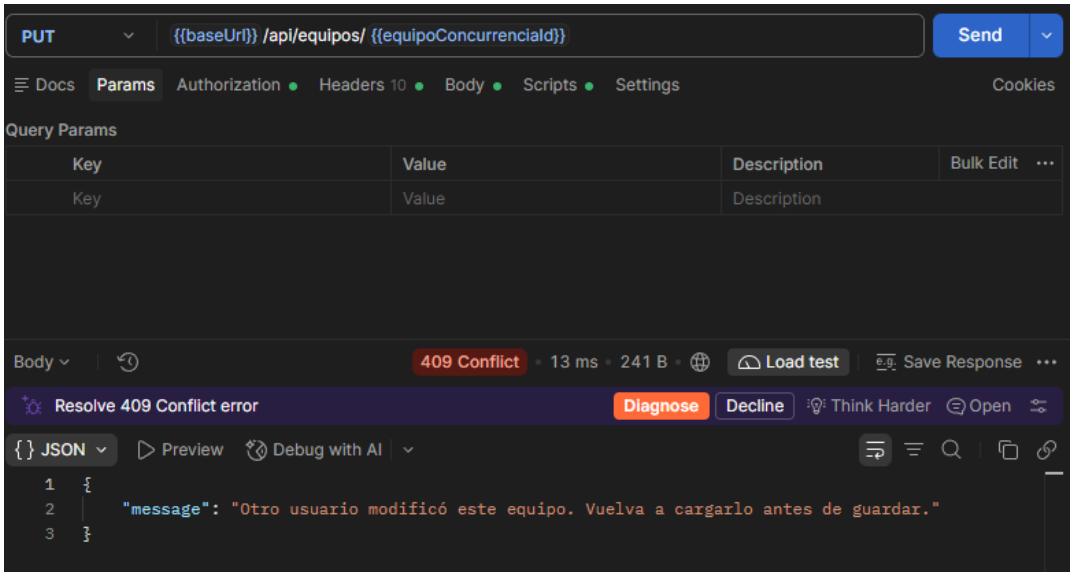
Captura 27.



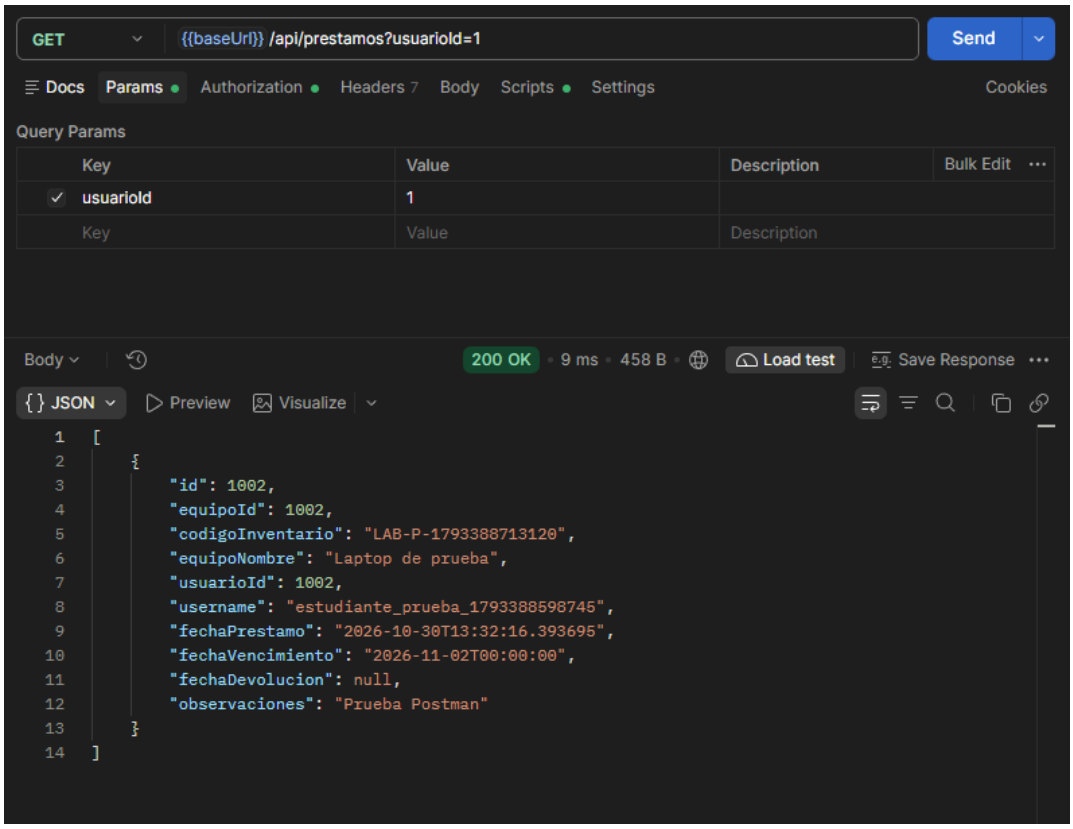
Captura 28.



Captura 29.



Captura 30.



Captura 31.